

LABORATÓRIO DE BANCO DE DADOS · UFMS

Da Modelagem ao MySQL

Festival Pantanal Sônico: revisão de
modelagem

2026/2

Prof. Dr. Márcio Silva

Faculdade de Computação

Universidade Federal de Mato Grosso do Sul



Roteiro de hoje

①

Minimundo → diagrama de Chen

Do enunciado ao modelo conceitual.

②

7 regras de mapeamento

Do diagrama de Chen ao modelo relacional.

③

DDL + carga no MySQL

Do modelo relacional ao banco rodando: criar, carregar, alterar, remover.

④

Consultas de sanidade

Do banco rodando à confirmação de que ele responde certo.

Objetivo de hoje

Reconstruir o caminho que separa **o enunciado** do **banco rodando**.

Por que modelar antes de programar?

SQL sem modelo é chute com sintaxe. O modelo é o contrato: **erro de modelagem custa ALTER TABLE em produção; erro de SQL custa um UPDATE.**



O minimundo

O Festival Pantanal Sônico acontece todo ano em Campo Grande/MS. O festival monta **palcos**, cada um com nome único, localização e capacidade. Cada palco tem **exatamente um** patrocinador master, e cada patrocinador (CNPJ, nome, ramo) banca **no máximo um** palco, pagando uma **cota** negociada por palco.

Todas as pessoas envolvidas na produção têm CPF, nome, data de nascimento (a **idade** interessa aos relatórios), **endereço** (rua, número, bairro, cidade, UF) e **um ou mais telefones**. Uma pessoa pode ser **artista** (nome artístico único, cachê) **ou técnico** (função, nº de registro), nunca os dois ao mesmo tempo; e há pessoas na produção que não são nem um nem outro.

Cada apresentação acontece num palco, numa data e horário de início, com uma duração prevista; dentro do mesmo palco, **data + hora identificam** a apresentação. Toda apresentação tem **um artista principal**; um artista pode fazer várias apresentações.

A equipe técnica de cada apresentação é formada por **vários técnicos**, e um técnico trabalha em **várias apresentações**; em cada escala registra-se o **turno** (montagem, show ou desmontagem).

Dicionário de símbolos: notação de Chen



Entidade forte

Ex.: PALCO



Entidade fraca (retângulo duplo)

Ex.: APRESENTACAO (fraca de PALCO)



Relacionamento (losango)

Ex.: PATROCINA



Relacionamento identificador (losango duplo)

Ex.: OCORRE_EM



Atributo (elipse)

Ex.: PALCO.nome



Atributo multivalorado (elipse dupla)

Ex.: PESSOA.telefones



Atributo derivado (elipse tracejada)

Ex.: PESSOA.idade



Atributo composto

Ex.: PESSOA.endereço



Atributo chave (sublinhado)

Ex.: PESSOA.cpf



Discriminante (sublinhado tracejado)

Ex.: APRESENTACAO em PALCO



Participação total (linha dupla)

Ex.: APRESENTACAO em OCORRE_EM



Participação parcial (linha simples)

Ex.: PATROCINADOR em PATROCINA

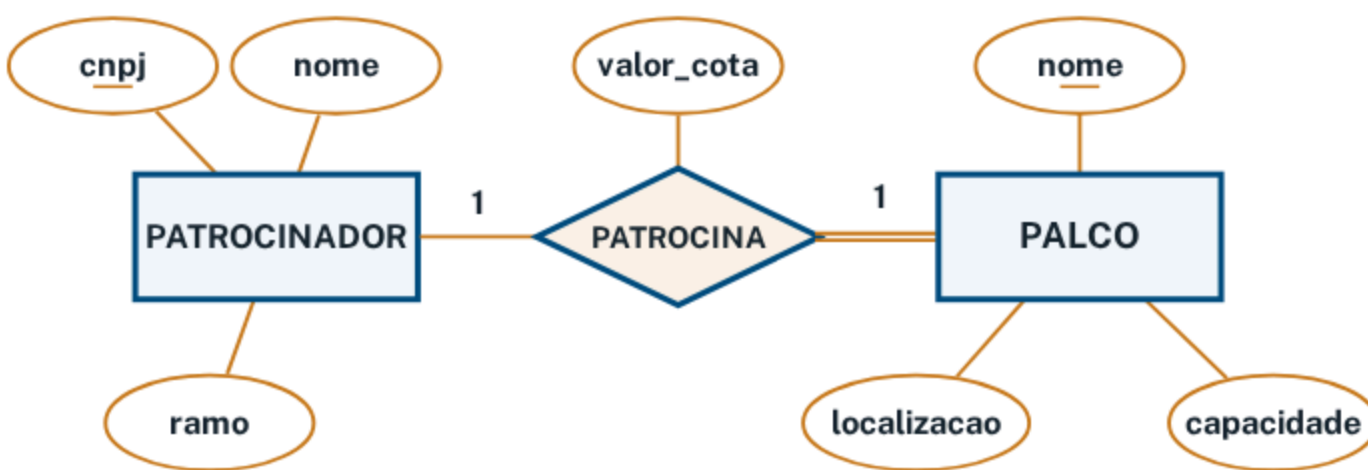


Rótulos de cardinalidade

Ex.: PALCO(1)-OCORRE_EM-APRESENTACAO(N)

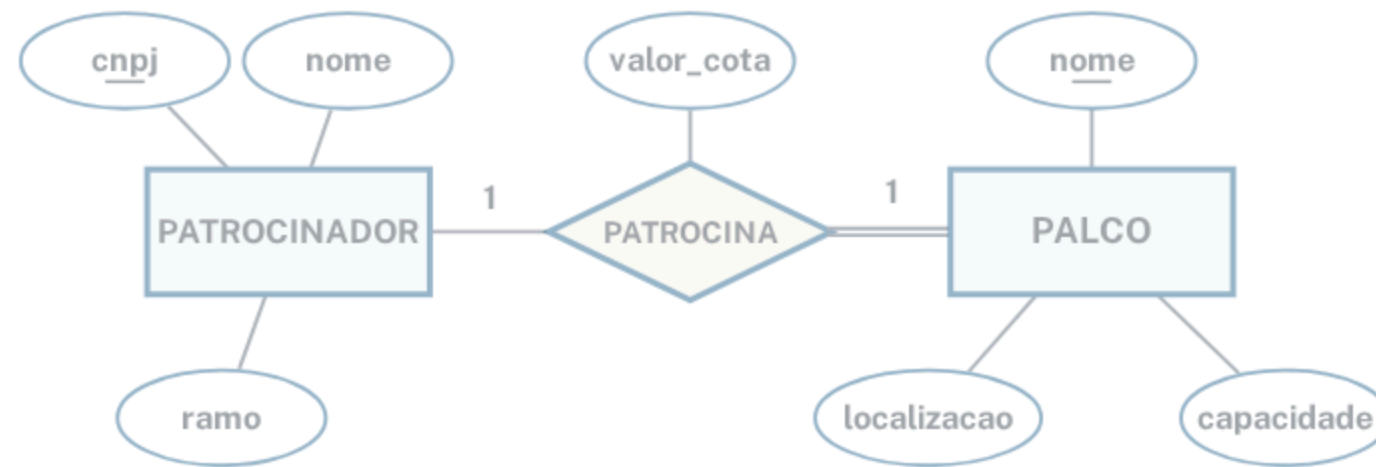
A cota é atributo de quem?

“Cada palco tem exatamente um patrocinador master, e cada patrocinador (CNPJ, nome, ramo) banca no máximo um palco, pagando uma cota negociada por palco.”



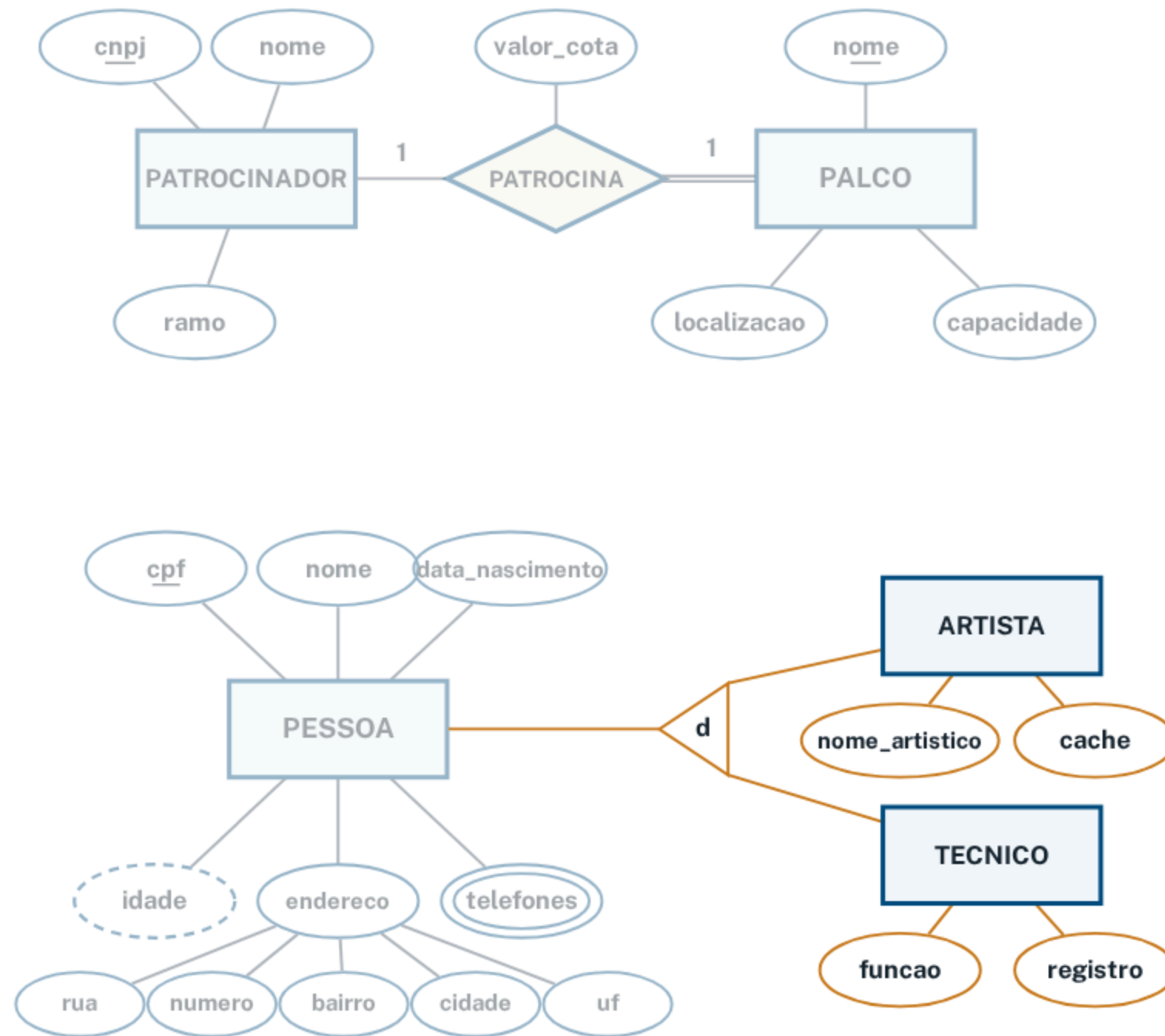
Por que idade é uma elipse tracejada?

“Todas as pessoas envolvidas na produção têm CPF, nome, data de nascimento (a idade interessa aos relatórios), endereço (rua, número, bairro, cidade, UF) e um ou mais telefones.”



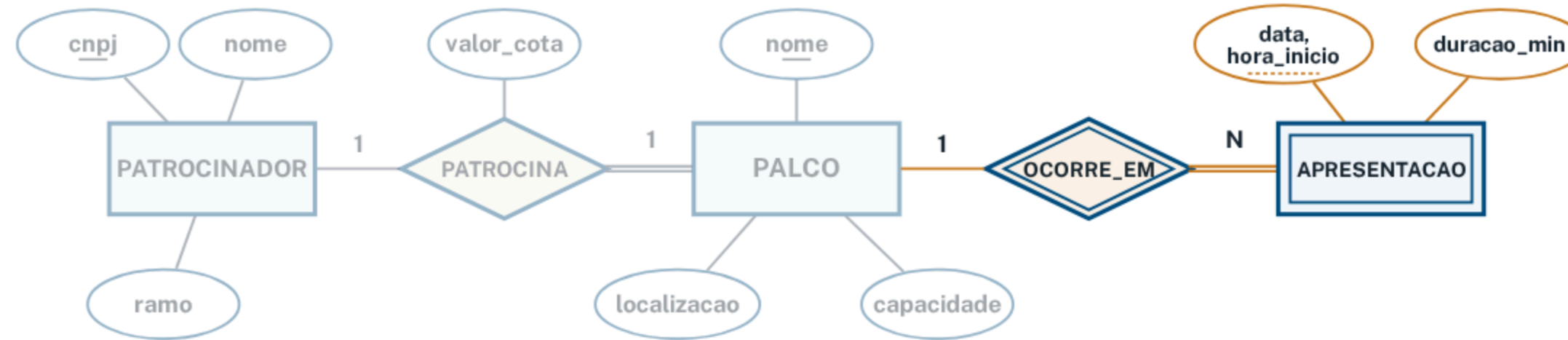
Helena, da produção, não é artista nem técnico. Onde ela fica?

“Uma pessoa pode ser artista (nome artístico único, cachê) ou técnico (função, nº de registro), nunca os dois ao mesmo tempo; e há pessoas na produção que não são nem um nem outro.”



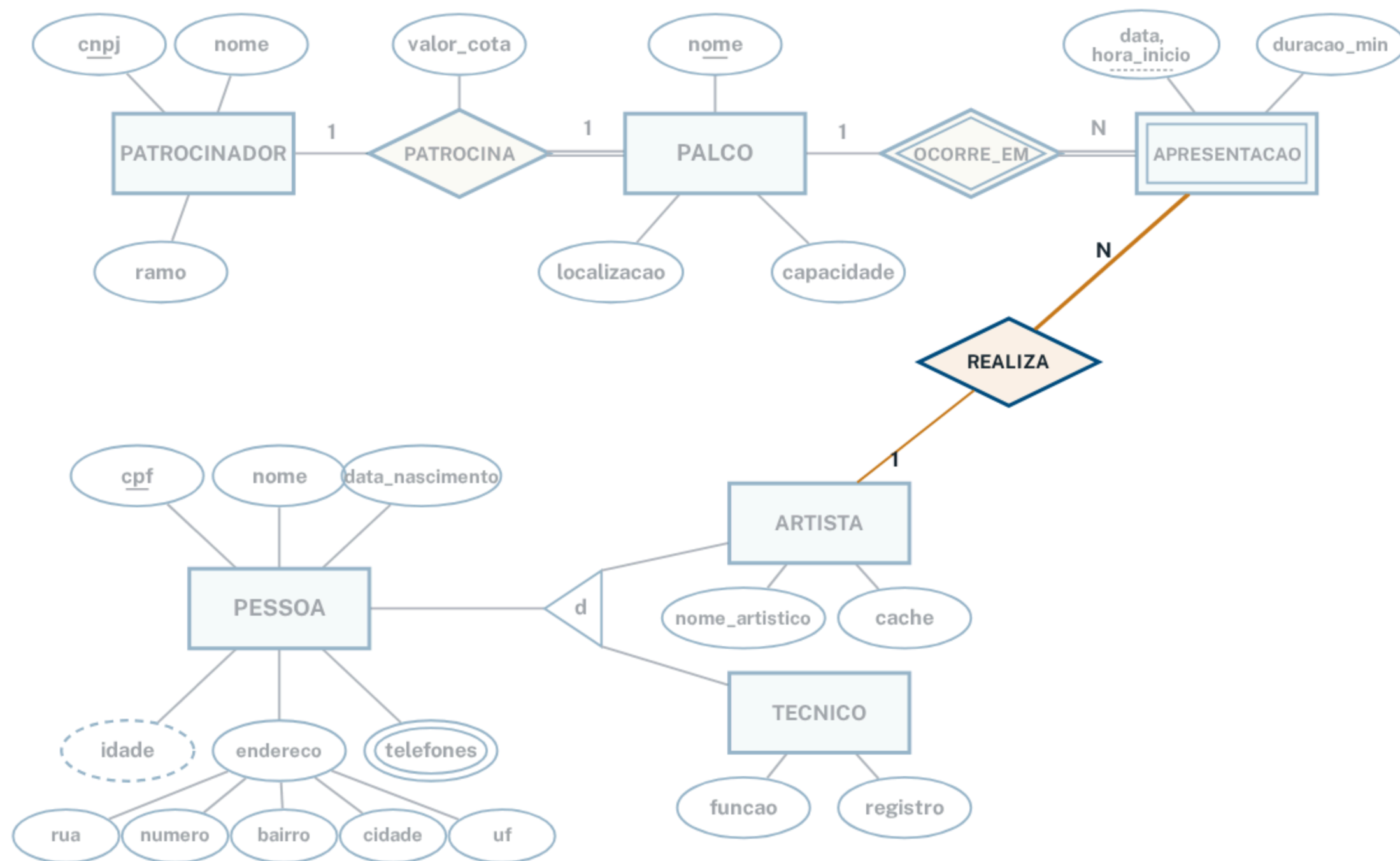
Por que data + hora não bastam como chave?

“Cada apresentação acontece num palco, numa data e horário de início, com uma duração prevista; dentro do mesmo palco, data + hora identificam a apresentação.”



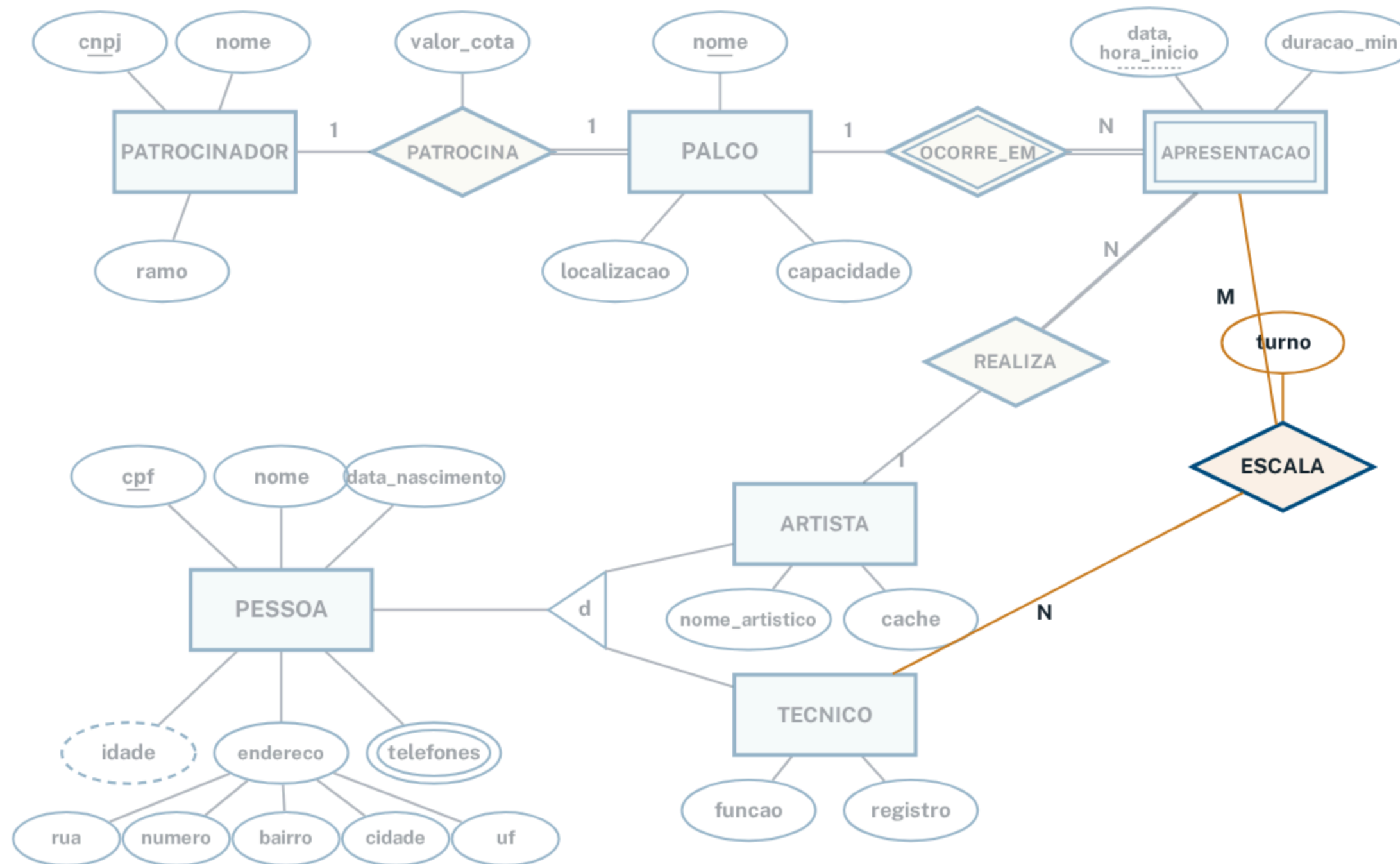
Todo show tem artista. E um artista, um só show?

“Toda apresentação tem um artista principal; um artista pode fazer várias apresentações.”



Turno pode ficar dentro de TECNICO? E de APRESENTACAO?

“A equipe técnica de cada apresentação é formada por vários técnicos, e um técnico trabalha em várias apresentações; em cada escala registra-se o turno (montagem, show ou desmontagem).”



Três erros comuns ao ler o minimundo

① Atributo de relacionamento vira atributo de entidade

Colocar valor_cota dentro de PALCO ou de PATROCINADOR. Mas a cota depende do **par** (palco, patrocinador); ela só existe no relacionamento PATROCINA.

② Multivalorado promovido a entidade sem necessidade

Transformar telefones numa entidade TELEFONE cheia de atributos próprios. Se é só uma lista de valores da pessoa, uma elipse dupla resolve, sem inventar relacionamento novo.

③ Chave inventada para entidade fraca

Dar um id_apresentacao autoincremento a APRESENTACAO. A chave real já existe: PALCO (dono) + discriminante (data, hora_inicio). Entidade fraca não precisa de chave própria.

Os três erros têm a mesma raiz: **ler o substantivo antes de perguntar de quem ele depende**. Sempre volte ao minimundo e pergunte “isso pertence a uma coisa só, ou a um par?”

CHECAGEM RÁPIDA COM A TURMA

Aponte no diagrama completo: participação total, participação parcial, atributo derivado e a chave da entidade fraca.

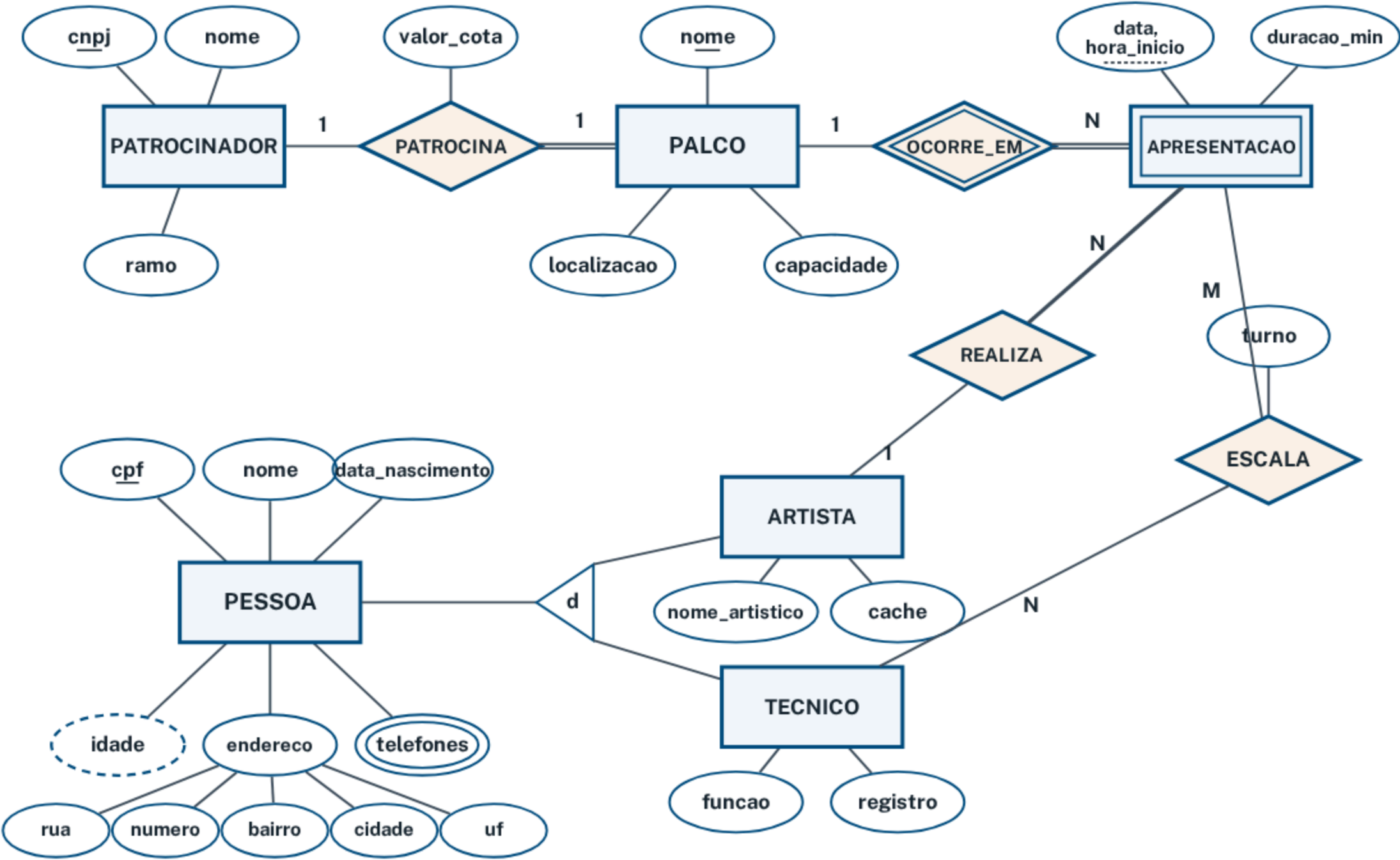
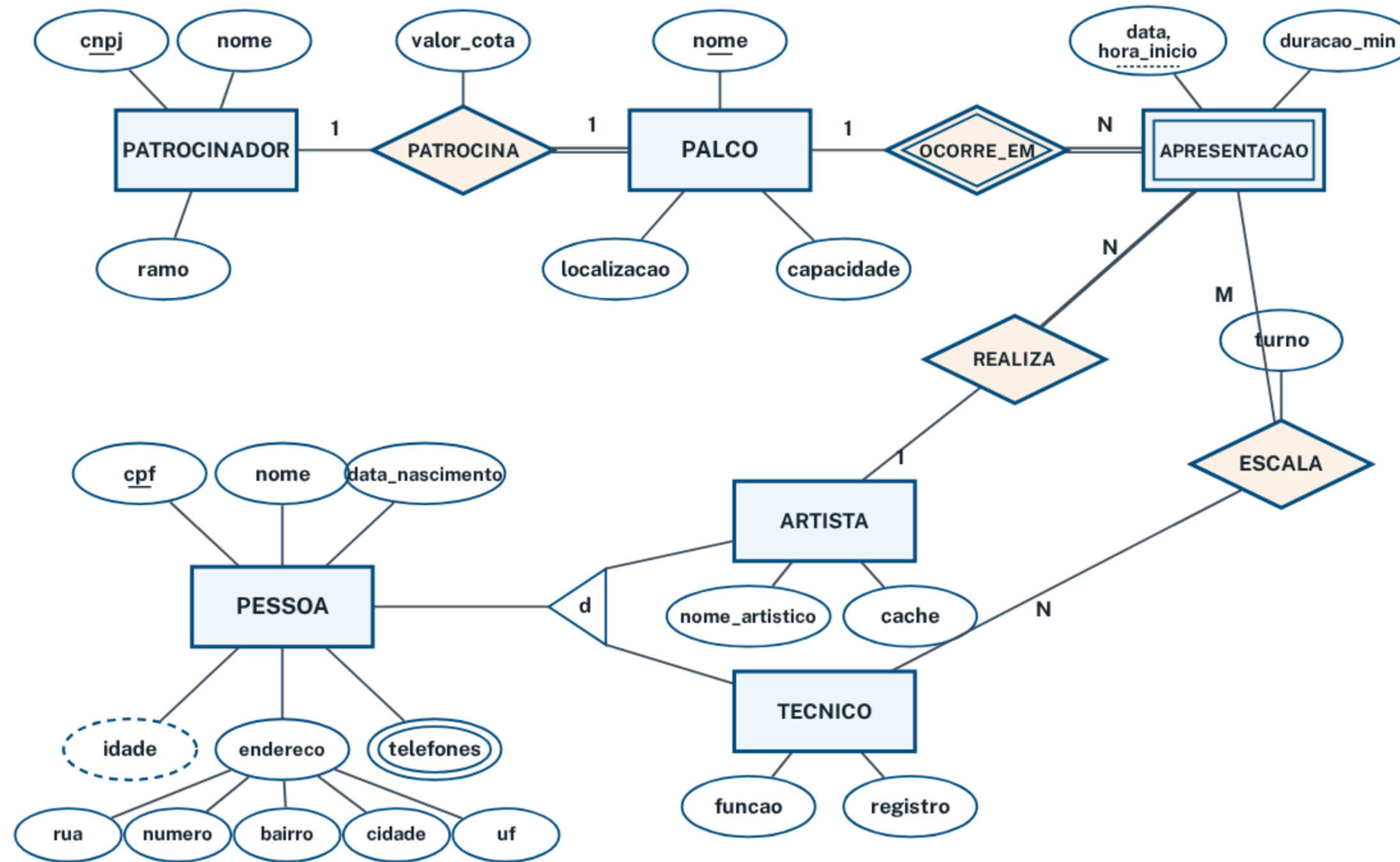


Diagrama completo: Festival Pantanal Sônico



Do diagrama de Chen ao relacional: sete regras

- R1**
Entidades fortes viram tabela
pessoa, patrocinador: PK = a chave da entidade.
- R2**
Entidades fracas viram tabela
apresentacao: PK = chave do dono + discriminante.
- R3**
Relacionamento 1:1
palco → patrocinador: FK + UNIQUE no lado de participação total.
- R4**
Relacionamento 1:N
apresentacao → artista: FK no lado N.
- R5**
Relacionamento N:M
escala_tecnica: tabela associativa, PK = as duas FKs.
- R6**
Atributo multivalorado
telefone_pessoa: tabela própria, PK = dono + valor.
- R7**
Especialização (ISA)
pessoa/artista/tecnico: superclasse + subclasses, PK = FK.

Por que essa ordem?

Fortes primeiro, porque todo mundo referencia elas: sem pessoa e patrocinador prontos, nenhuma FK tem para onde apontar. **Fraca logo depois do dono**: apresentacao só ganha PK porque palco (R1) já existe. **N:M por último**, porque escala_tecnica (R5) precisa dos dois lados já prontos: apresentacao (R2+R4) e tecnico (R7).

R1: Entidades fortes viram tabela



pessoa

(cpf, nome, data_nascimento,
end_rua, end_numero, end_bairro,
end_cidade, end_uf)

endereço (composto) **achata** em 5 colunas end_*. idade (derivado) **não** vira coluna. Idade?
Paciência, slide S3.

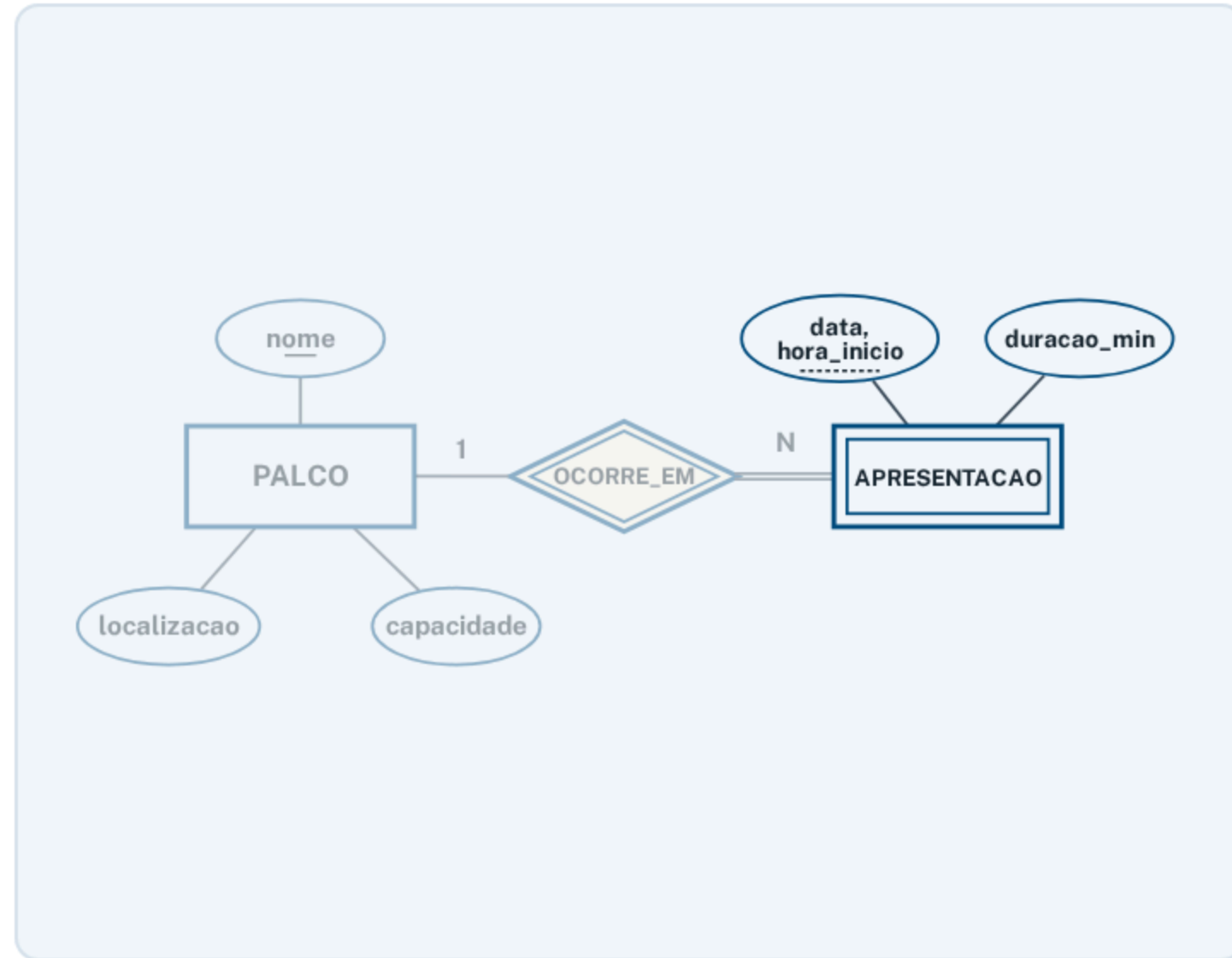
patrocinador

(cnpj, nome, ramo)

Sem composto, sem derivado: entidade forte simples, direto pro CREATE TABLE.

Toda **entidade forte** vira uma tabela; atributo composto achata em colunas, atributo derivado nunca vira coluna.

R2: Entidades fracas viram tabela



apresentacao

(nome_palco → palco, data, hora_inicio,
duracao_min, ...)

A FK para o dono (nome_palco) **já nasce dentro da PK**; ela não é um campo extra, é parte da identidade da apresentação.

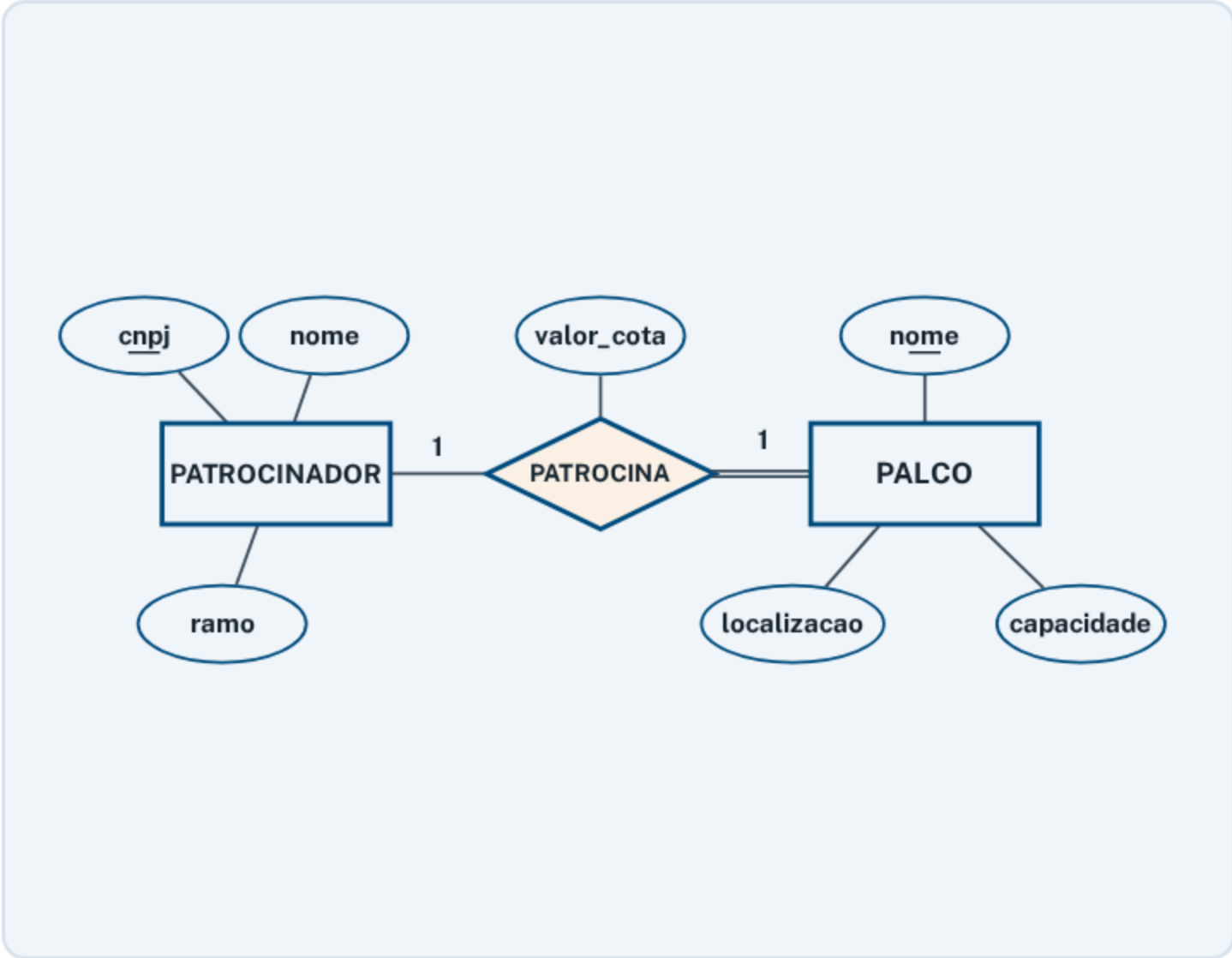
palco

(nome, ...)

o dono, já pronto pela R1/R3; a chave é o nome (único no minimundo).

Toda **entidade fraca** vira tabela com PK = chave do dono + discriminante; a FK para o dono entra dentro da própria PK.

R3: Relacionamento 1:1



palco

(nome, localizacao, capacidade,
cnpj_patrocinador → patrocinador **UNIQUE**,
 valor_cota)

PK = nome, a **chave natural** (“nome único” no minimundo). valor_cota (atributo do relacionamento PATROCINA) mora junto da FK, no mesmo lado.

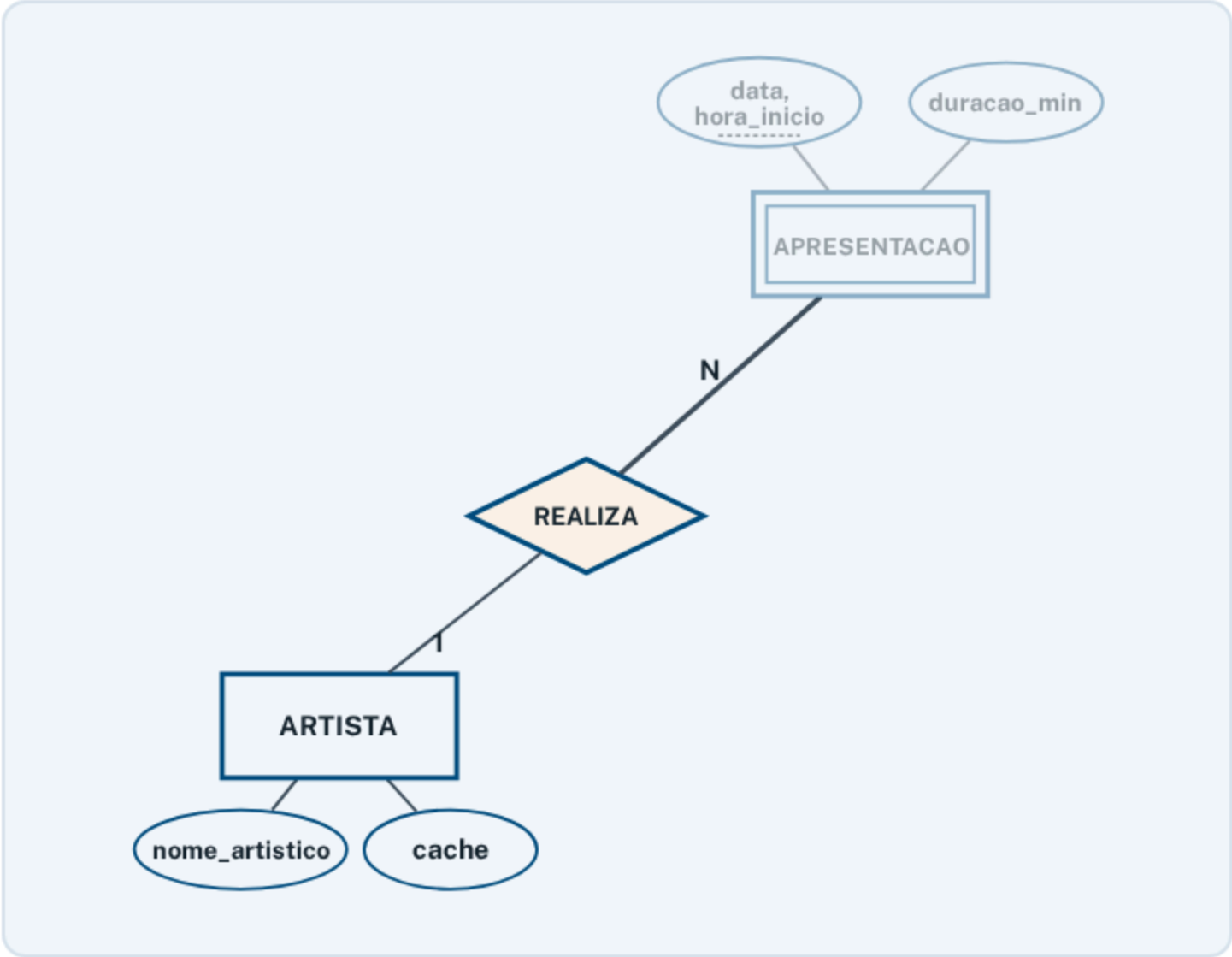
patrocinador

(cnpj, nome, ramo)

sem FK: não sabe (nem precisa saber) qual palco.

FK + UNIQUE = a jogada inteira. Por que no palco e não no patrocinador? Participação total: **todo palco tem patrocinador**, nem todo patrocinador tem palco.

R4: Relacionamento 1:N



apresentacao

(nome_palco → palco, data, hora_inicio,
duracao_min, cpf_artista → artista)

A FK vive no lado **N** do relacionamento (muitas apresentações, um artista principal cada).

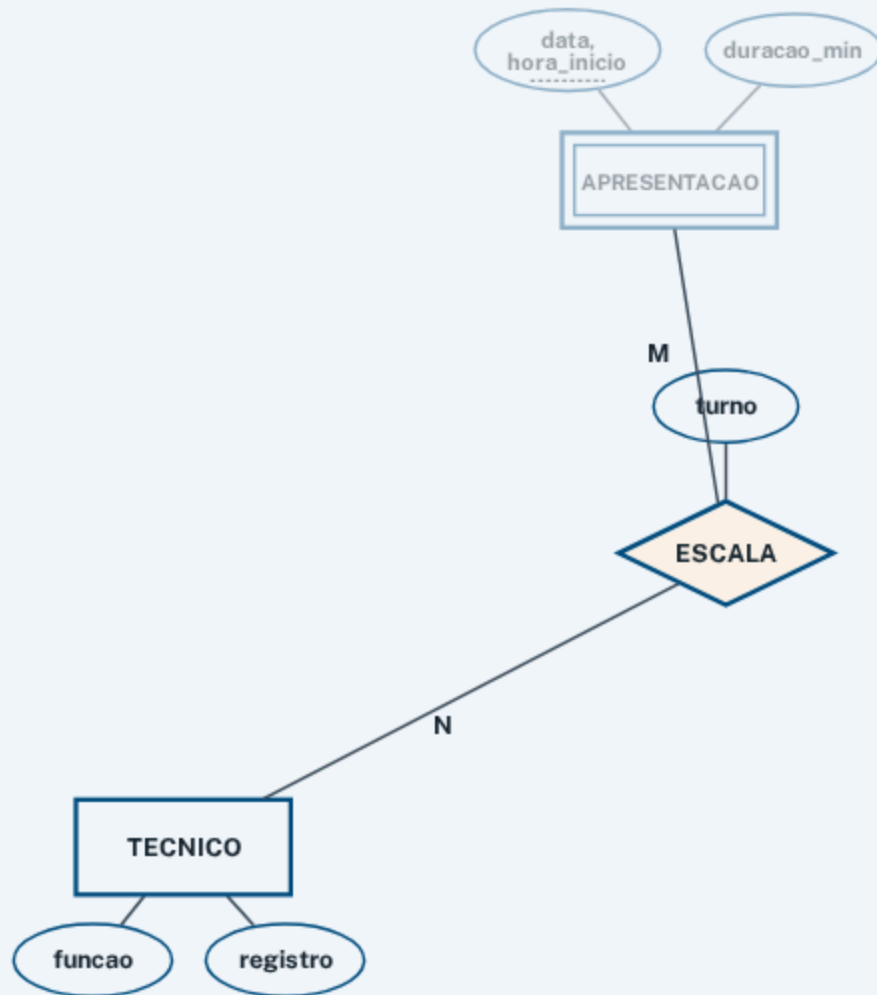
artista

(cpf → pessoa, ...)

o lado 1, sem FK extra para apresentacao.

Todo relacionamento 1:N vira uma FK no lado N apontando para a chave do lado 1, nunca o contrário.

R5: Relacionamento N:M



escala_tecnica

(nome_palco, data, hora_inicio → apresentacao,
cpf_tecnico → tecnico,
 turno)

PK = as **duas FKs** juntas (a de apresentacao é composta: 3 colunas). turno (atributo do relacionamento) vira coluna comum.

Todo relacionamento N:M vira tabela associativa própria; a PK é a junção das FKs dos dois lados.

R6: Atributo multivalorado



telefone_pessoa

(cpf → pessoa, telefone)

PK = dono + valor. Cada telefone é uma **linha**, não uma coluna; é assim que "um ou mais telefones" cabe numa tabela relacional.

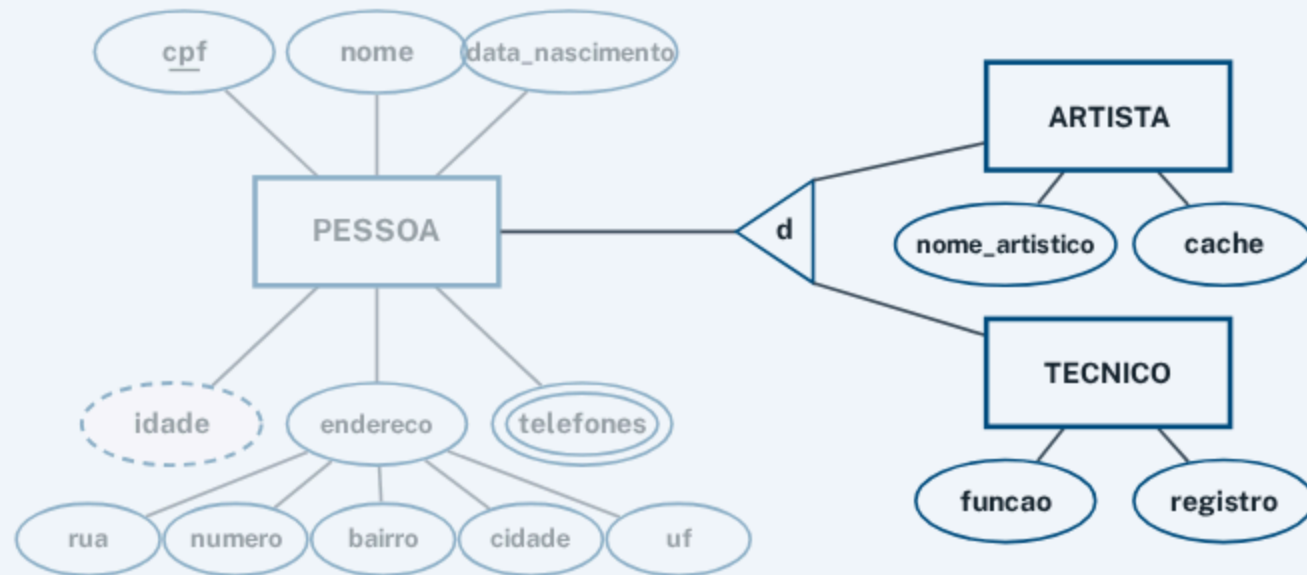
pessoa

(cpf, ...)

o dono, sem coluna "telefones" nenhuma.

Todo **atributo multivalorado** vira tabela própria; a PK é o dono + o próprio valor (não precisa de id sintético).

R7: O problema da especialização



Uma tabela? Duas? Três?

PESSOA é a superclasse; ARTISTA e TECNICO são subclasses de uma especialização **disjunta** (d) e **parcial** (Helena não é nem um nem outro). Ao contrário das regras R1–R6, aqui **não existe** um único jeito certo de mapear: existem estratégias, com trade-offs diferentes.

Próximo slide: as 3 estratégias possíveis, e por que escolhemos uma delas para o Festival Pantanal Sônico.

R7: Três estratégias para a especialização

(a) Tabela única + discriminador

`pessoa(cpf, nome, ..., tipo,
 nome_artístico, cache, funcao,
 registro)`

Uma coluna tipo diz se é artista ou técnico. Os atributos exclusivos de cada subclasse ficam **NULL** para quem não é daquele tipo; para a Helena, os dois grupos de colunas ficam NULL.

(b) Uma tabela por subclasse

`artista(cpf, nome, ...,
 nome_artístico, cache)
 tecnico(cpf, nome, ..., funcao,
 registro)`

Repete os atributos herdados de PESSOA em cada subclasse. Só funciona se a especialização for **total** (toda instância cai em alguma subclasse) e **disjunta**. A nossa é parcial: **Helena ficaria sem lugar nenhum**.

(c) Superclasse + subclasses, PK = FK

`pessoa(cpf, nome, ...)
 artista(cpf → pessoa,
 nome_artístico, cache)
 tecnico(cpf → pessoa, funcao,
 registro)`

A PK da subclasse é também FK para a superclasse. Funciona para especialização **parcial**: quem não é subclasse simplesmente não tem linha em nenhuma das duas.

Veredito: escolhemos (c): sem NULLs espalhados, Helena vive tranquila em pessoa sem entrar em nenhuma subclasse, e o preço que se paga é um JOIN quando se quer os dados completos de um artista ou técnico.

Esquema relacional completo: 8 tabelas



Rótulo em cada canto = a regra que originou a tabela. apresentacao nasce de duas regras: R2 (entidade fraca) dá a PK composta, R4 (1:N) acrescenta cpf_artista.

O comando CREATE TABLE: anatomia

forma geral

```
CREATE TABLE nome_da_tabela (
  coluna1 TIPO NOT NULL,          -- coluna obrigatória
  coluna2 TIPO,                  -- sem NOT NULL: aceita NULL
  CONSTRAINT pk_tabela           PRIMARY KEY (coluna1),
  CONSTRAINT uq_tabela_col2      UNIQUE (coluna2),
  CONSTRAINT fk_tabela_outra     FOREIGN KEY (coluna2)
    REFERENCES outra_tabela (sua_pk),
  CONSTRAINT ck_tabela_regra     CHECK (coluna1 > 0)
) ENGINE = InnoDB;
```

tipos que vamos usar

```
CHAR(n)      -- tamanho FIXO: cpf, cnpj, uf
VARCHAR(n)   -- tamanho variável: nomes, endereços
INT           -- inteiro: capacidade, duração
DECIMAL(p,d) -- exato, p dígitos (d decimais): dinheiro
DATE         -- '2026-09-18'
TIME         -- '20:00:00'
ENUM('a','b')-- domínio fechado de valores
```

Cada regra vira uma **constraint com nome**: prefixos pk_, fk_, uq_, ck_. É esse nome que aparece na mensagem quando o banco recusa uma linha; constraint anônima vira erro anônimo.

Entidades fortes: pessoa e patrocinador

pessoa

```
-- -----
-- Regra 1: Entidades fortes
-- Atributo composto (endereço) é achatado em colunas.
-- Atributo derivado (idade) NÃO vira coluna: é calculado na
-- consulta.
-- -----
```

```
CREATE TABLE pessoa (
  cpf          CHAR(11)    NOT NULL,
  nome         VARCHAR(100) NOT NULL,
  data_nascimento DATE     NOT NULL,
  end_rua      VARCHAR(80) NOT NULL,
  end_numero   VARCHAR(10) NOT NULL,
  end_bairro   VARCHAR(60) NOT NULL,
  end_cidade   VARCHAR(60) NOT NULL,
  end_uf       CHAR(2)     NOT NULL,
  CONSTRAINT pk_pessoa PRIMARY KEY (cpf)
) ENGINE = InnoDB;
```

patrocinador

```
CREATE TABLE patrocinador (
  cnpj CHAR(14)    NOT NULL,
  nome VARCHAR(100) NOT NULL,
  ramo VARCHAR(50) NOT NULL,
  CONSTRAINT pk_patrocinador PRIMARY KEY (cnpj)
) ENGINE = InnoDB;
```

Composto achatado, derivado ausente: end_rua...end_uf são o atributo composto endereço (R1) virando 5 colunas; **idade não existe em nenhuma coluna**; volta na S3, no fim do deck, calculada, não armazenada.

Relacionamento 1:1: palco patrocinado

```
-- -----
-- Regra 3: Relacionamento 1:1 (PATROCINA)
-- O minimundo diz "nome único" → nome é a CHAVE NATURAL do palco
-- (nada de id artificial: a chave já existe no próprio minimundo).
-- FK fica no lado de participação TOTAL (todo palco tem patrocinador).
-- UNIQUE na FK transforma o 1:N que a FK criaria em 1:1.
-- O atributo do relacionamento (valor_cota) acompanha a FK.
-- -----
```

```
CREATE TABLE palco (
  nome          VARCHAR(50)  NOT NULL,
  localizacao   VARCHAR(100) NOT NULL,
  capacidade     INT          NOT NULL,
  cnpj_patrocinador CHAR(14)  NOT NULL,
  valor_cota     DECIMAL(12,2) NOT NULL,
  CONSTRAINT pk_palco PRIMARY KEY (nome),
  CONSTRAINT uq_palco_patrocinador UNIQUE (cnpj_patrocinador),
  CONSTRAINT fk_palco_patrocinador FOREIGN KEY (cnpj_patrocinador)
    REFERENCES patrocinador (cnpj),
  CONSTRAINT ck_palco_capacidade CHECK (capacidade > 0),
  CONSTRAINT ck_palco_cota CHECK (valor_cota >= 0)
) ENGINE = InnoDB;
```

■ FK + UNIQUE: o par que transforma 1:N em 1:1 ■ CHECK: sanidade de valor

A FK cnpj_patrocinador sozinha já daria 1:N (vários palcos, um patrocinador); a UNIQUE na mesma coluna é o que trava em 1:1: cada patrocinador financia no máximo um palco.

Especialização (estratégia c) e multivalorado

artista

```

-----
-----
-- Regra 7: Especialização (estratégia c:
superclasse + subclasses,
-- PK da subclasse é também FK para a
superclasse)
-----
-----

CREATE TABLE artista (
  cpf          CHAR(11)      NOT NULL,
  nome_artistico VARCHAR(80)  NOT NULL,
  cache        DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_artista PRIMARY KEY (cpf),
  CONSTRAINT uq_artista_nome UNIQUE
(nome_artistico),
  CONSTRAINT fk_artista_pessoa FOREIGN KEY
(cpf) REFERENCES pessoa (cpf),
  CONSTRAINT ck_artista_cache CHECK (cache
>= 0)
) ENGINE = InnoDB;

```

tecnico

```

CREATE TABLE tecnico (
  cpf          CHAR(11)      NOT NULL,
  funcao       VARCHAR(50)  NOT NULL,
  registro     VARCHAR(20)  NOT NULL,
  CONSTRAINT pk_tecnico PRIMARY KEY (cpf),
  CONSTRAINT uq_tecnico_registro UNIQUE
(registro),
  CONSTRAINT fk_tecnico_pessoa FOREIGN KEY
(cpf) REFERENCES pessoa (cpf)
) ENGINE = InnoDB;

```

telefone_pessoa

```

-----
-----
-- Regra 6: Atributo multivalorado vira
tabela própria;
-- PK composta (dono + valor)
-----
-----

CREATE TABLE telefone_pessoa (
  cpf          CHAR(11)      NOT NULL,
  telefone     VARCHAR(20)  NOT NULL,
  CONSTRAINT pk_telefone_pessoa PRIMARY KEY
(cpf, telefone),
  CONSTRAINT fk_telefone_pessoa FOREIGN KEY
(cpf) REFERENCES pessoa (cpf)
) ENGINE = InnoDB;

```

Ler o destaque

cpf é **PK e FK ao mesmo tempo** em artista e tecnico, assinatura da estratégia (c): a identidade da subclasse é a identidade da superclasse.

Em telefone_pessoa, a PK é **composta** (dono + valor), nenhuma coluna sintética de id.

Entidade fraca e associativa N:M

apresentacao

```
-- -----
-- Regra 2: Entidade fraca. PK = chave do dono + discriminante.
-- Regra 4: Relacionamento 1:N (REALIZA), FK no lado N.
-- -----

CREATE TABLE apresentacao (
  nome_palco VARCHAR(50) NOT NULL,
  data DATE NOT NULL,
  hora_inicio TIME NOT NULL,
  duracao_min INT NOT NULL,
  cpf_artista CHAR(11) NOT NULL,
  CONSTRAINT pk_apresentacao PRIMARY KEY (nome_palco, data,
hora_inicio),
  CONSTRAINT fk_apresentacao_palco FOREIGN KEY (nome_palco)
    REFERENCES palco (nome),
  CONSTRAINT fk_apresentacao_artista FOREIGN KEY (cpf_artista)
    REFERENCES artista (cpf),
  CONSTRAINT ck_apresentacao_duracao CHECK (duracao_min BETWEEN 15
AND 300)
) ENGINE = InnoDB;
```

escala_tecnica

```
-- -----
-- Regra 5: Relacionamento N:M vira tabela associativa;
-- PK = FKs dos dois lados; atributo do relacionamento (turno) vira
coluna
-- -----

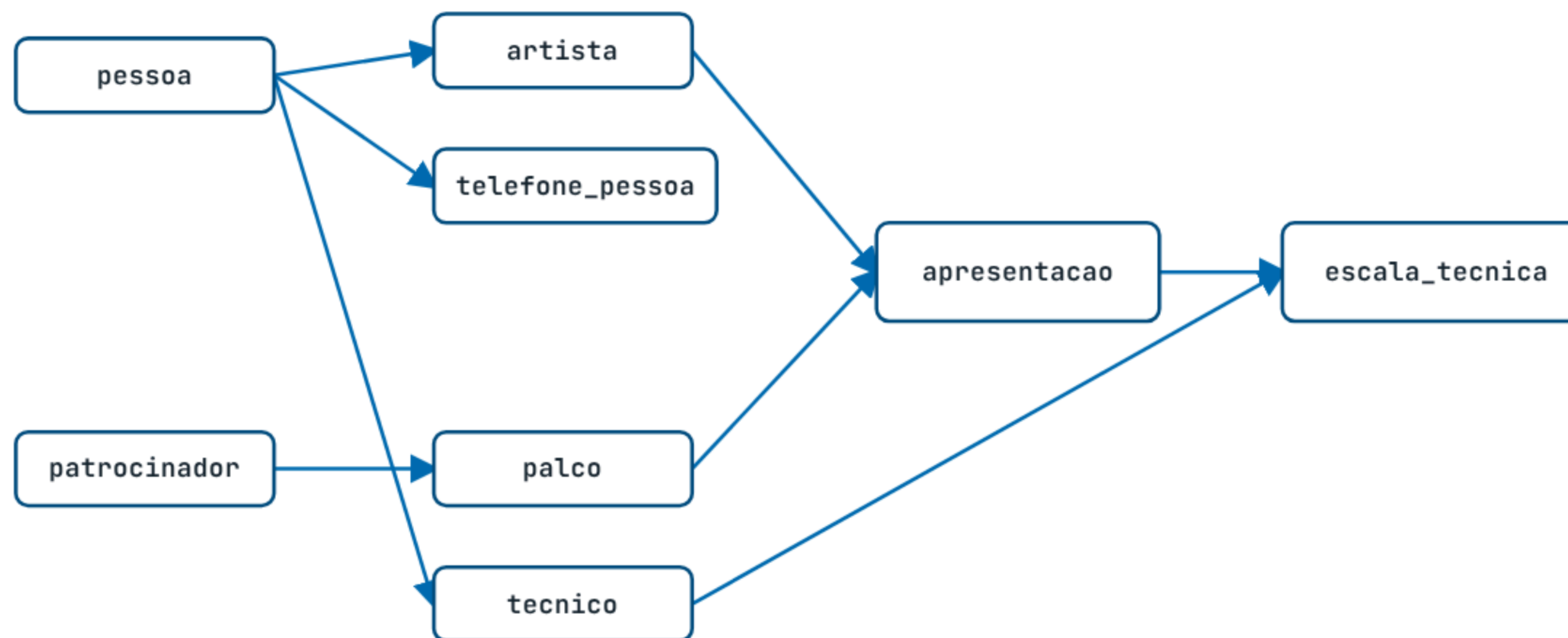
CREATE TABLE escala_tecnica (
  nome_palco VARCHAR(50) NOT NULL,
  data DATE NOT NULL,
  hora_inicio TIME NOT NULL,
  cpf_tecnico CHAR(11) NOT NULL,
  turno ENUM('montagem','show','desmontagem') NOT NULL,
  CONSTRAINT pk_escala PRIMARY KEY (nome_palco, data, hora_inicio,
cpf_tecnico),
  CONSTRAINT fk_escala_apresentacao FOREIGN KEY (nome_palco, data,
hora_inicio)
    REFERENCES apresentacao (nome_palco, data, hora_inicio),
  CONSTRAINT fk_escala_tecnico FOREIGN KEY (cpf_tecnico)
    REFERENCES tecnico (cpf)
) ENGINE = InnoDB;
```

■ PK composta / FK composta ■ ENUM: domínio fechado de valores

apresentacao nasce fraca (PK = dono + discriminante); escala_tecnica referencia essa PK **composta inteira** como FK (3 colunas); é assim que uma associativa aponta pra uma entidade fraca.

ANTES DE INSERIR UMA LINHA

A ordem da carga: grafo de dependências (FKs)



seta = precisa existir antes



Na P2 de 2026/1, **7 entregas caíram exatamente aqui**: dump com FK fora de ordem não executa.

O comando INSERT: dois formatos

com colunas nomeadas (o que usamos)

```
INSERT INTO tabela (col_a, col_b) VALUES  
( 'valor 1', 10),  
( 'valor 2', 20),  
( 'valor 3', 30);
```

Você escolhe **quais** colunas e em **que ordem**; o que ficar de fora recebe DEFAULT ou NULL. Várias linhas num comando só: tuplas separadas por vírgula.

Nomeie as colunas sempre: o INSERT sobrevive a mudança de esquema e quem lê sabe o que cada valor é. Toda a carga do festival, no próximo slide, usa o formato nomeado.

sem colunas nomeadas

```
INSERT INTO tabela VALUES  
( 'valor 1', 10, NULL, 'x');
```

Exige **todos os valores, na ordem exata** em que as colunas aparecem no CREATE TABLE. Se a tabela ganhar uma coluna, o comando quebra. Pior: pode inserir valor na coluna errada sem avisar.

Dois recortes de INSERT

palco (3 linhas)

```
INSERT INTO palco (nome, localizacao, capacidade,
cnpj_patrocinador, valor_cota) VALUES
('Palco Aquidauana', 'Gramado central',      8000,
'000000000000101', 250000.00),
('Palco Cordilheira', 'Arena coberta',        3500,
'000000000000102', 180000.00),
('Palco Nascente',   'Deck do lago',          1200,
'000000000000103',  90000.00);
```

apresentacao (6 linhas)

```
INSERT INTO apresentacao (nome_palco, data, hora_inicio,
duracao_min, cpf_artista) VALUES
('Palco Aquidauana', '2026-09-18', '20:00:00', 90,
'11111111101'),
('Palco Aquidauana', '2026-09-18', '22:00:00', 75,
'11111111103'),
('Palco Aquidauana', '2026-09-19', '21:00:00', 90,
'11111111106'),
('Palco Cordilheira', '2026-09-18', '19:30:00', 60,
'11111111106'),
('Palco Cordilheira', '2026-09-19', '20:30:00', 80,
'11111111101'),
('Palco Nascente',   '2026-09-19', '18:00:00', 45,
'11111111103');
```

Contagem real após o script inteiro rodar (8 tabelas, ordem do grafo de dependências):

pessoa

8

linhas

apresentacao

6

linhas

escala_tecnica

10

linhas

Os comandos UPDATE e DELETE

UPDATE: altera linhas existentes

```
UPDATE tabela
SET  col_a = novo_valor,
     col_b = col_b * 1.10  -- pode usar o valor atual
WHERE condição;
```

SET diz **o que** muda; WHERE diz **em quais linhas**. A expressão do SET pode ler a própria coluna (reajuste, acréscimo, desconto).

Os dois comandos **sem WHERE atingem a tabela inteira**: o MySQL executa sem perguntar nada. Escreva o WHERE primeiro, de preferência pela chave primária.

DELETE: remove linhas inteiras

```
DELETE FROM tabela
WHERE condição;
```

Remove a **linha inteira** que casa com o WHERE; não existe "apagar só uma coluna" (isso é UPDATE ... SET col = NULL).

UPDATE E DELETE EM AÇÃO

Reajustes e cortes no festival

U1: palco ampliado · U2: cachê reajustado

```
-- U1) Obra concluída: Palco Aquidauana agora comporta 9000
UPDATE palco
SET capacidade = 9000
WHERE nome = 'Palco Aquidauana';
-- Query OK, 1 row affected

-- U2) Aurora renegociou: +10% sobre o cachê ATUAL
UPDATE artista
SET cache = cache * 1.10
WHERE nome_artístico = 'Aurora do Pantanal';
-- Query OK, 1 row affected (45000.00 → 49500.00)
```

WHERE pela chave: nome em palco, nome_artístico (UNIQUE) em artista: 1 linha cada, nunca mais que isso.

D1: técnico sai da escala · D2: FK impede apagar o palco

```
-- D1) Estevão saiu do show do Palco Nascente
DELETE FROM escala_tecnica
WHERE nome_palco = 'Palco Nascente'
AND data = '2026-09-19'
AND hora_inicio = '18:00:00'
AND cpf_tecnico = '1111111104';
-- Query OK, 1 row affected

-- D2) E apagar o palco inteiro?
DELETE FROM palco
WHERE nome = 'Palco Nascente';
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
(`festival`.`apresentacao`, CONSTRAINT `fk_apresentacao_palco` FOREIGN KEY (`nome_palco`)
REFERENCES `palco` (`nome`))
```

D1 passa porque o WHERE usa a **PK completa** da escala (4 colunas). D2 é barrado: o palco ainda tem apresentações ligadas pela FK, e o banco impede a remoção. É disso que trata o próximo slide.

QUANDO A MODELAGEM FALHA, O BANCO AVISA

Quando o banco recusa a linha: 3 erros reais

UNIQUE · RELACIONAMENTO 1:1

```
INSERT INTO palco (nome, localizacao, capacidade,
  cnpj_patrocinador, valor_cota)
VALUES ('Palco Pirata', 'x', 100,
  '000000000000101', 1.00);
```

ERROR 1062 (23000) at line 1: Duplicate entry
'000000000000101' for key 'palco.uq_palco_patrocinador'

FOREIGN KEY

```
INSERT INTO apresentacao (nome_palco, data,
  hora_inicio, duracao_min, cpf_artista)
VALUES ('Palco Aquidauana', '2026-09-20',
  '20:00:00', 60, '999999999999');
```

ERROR 1452 (23000) at line 1: Cannot add or update a
child row: a foreign key constraint fails
(`festival`.`apresentacao`, CONSTRAINT
`fk\_apresentacao\_artista` FOREIGN KEY (`cpf\_artista`)
REFERENCES `artista` (`cpf`))

CHECK

```
INSERT INTO palco (nome, localizacao, capacidade,
  cnpj_patrocinador, valor_cota)
VALUES ('Palco Teste', 'x', 0,
  '000000000000201', 1.00);
```

ERROR 3819 (HY000) at line 1: Check constraint
'ck_palco_capacidade' is violated.

Modelagem certa transforma **erro silencioso em erro de banco**: sem UNIQUE/FK/CHECK, essas três linhas entrariam quietas, e a inconsistência só apareceria bem mais tarde, num relatório errado.

O DESTINO DO FILHO É CONFIGURÁVEL

FK com ação referencial: o que acontece com o filho?

```
FOREIGN KEY (nome_palco) REFERENCES palco (nome) ON UPDATE <ação> ON DELETE <ação>
```

```
-- Cenário dos cartões: o pai muda de nome, e o filho aponta para ele
UPDATE palco SET nome = 'Palco Lago' WHERE nome = 'Palco Nascente';
```

RESTRICT padrão

Pai com filho não muda nem sai: o MySQL **recusa a operação** na hora (ERROR 1451). Sem cláusula, é isso que vale.

PAI	Palco Nascente	recusado
FILHO	Palco Nascente	intacto

NO ACTION

No MySQL/InnoDB é **sinônimo de RESTRICT**: mesma recusa, mesmo erro. Em outros SGBDs a checagem pode ficar para o fim da transação.

PAI	Palco Nascente	recusado
FILHO	Palco Nascente	intacto

CASCADE

A operação **se propaga ao filho**:
rename no pai renomeia a FK do filho;
DELETE no pai apaga o filho junto.

PAI	→ Palco Lago	mudou
FILHO	→ Palco Lago	acompanha

SET NULL

O pai muda ou sai e o filho fica, mas **a FK vira NULL**: o vínculo se desfaz.
Exige coluna que aceite NULL.

PAI	→ Palco Lago	mudou
FILHO	NULL	órfão

Nosso schema não declara ação nenhuma, por isso o D2 caiu no RESTRICT. CASCADE combina com chave natural (renomear o palco propagaria o nome novo). SET NULL nem compila aqui: nome_palco é parte da PK (NOT NULL) e o MySQL recusa a FK na criação (ERROR 1830).

O comando SELECT: anatomia da consulta

forma geral

```
SELECT t.coluna, o.coluna AS apelido -- 4º: o que sai
FROM tabela t                       -- 1º: de onde parte
JOIN outra o ON o.pk = t.fk         -- 2º: junta FK → PK
WHERE condição                      -- 3º: filtra linhas
ORDER BY o.coluna;                  -- 5º: ordena o resultado
```

como ler

```
-- O banco executa na ordem dos números,
-- não na ordem em que se escreve.

tabela t      -- t é APELIDO (alias): encurta
               -- e desambigua colunas homônimas

JOIN ... ON   -- liga a FK de um lado à PK
               -- do outro; um JOIN por "salto"

AS apelido    -- renomeia a coluna na saída
```

JOIN percorre no sentido inverso o caminho que as FKs criaram: devolve às linhas os dados que a modelagem separou em tabelas. As três consultas a seguir são só isso, encadeado.

Line-up por palco



```
-- S1) Line-up por palco (JOIN encadeado até a superclasse)
-- Bônus da chave natural: o nome do palco já está na FK, sem JOIN com palco.
SELECT a.nome_palco AS palco, a.data, a.hora_inicio,
       ar.nome_artístico, p.nome AS nome_civil
FROM apresentacao a
JOIN artista ar ON ar.cpf = a.cpf_artista
JOIN pessoa p ON p.cpf = ar.cpf
ORDER BY a.nome_palco, a.data, a.hora_inicio;
```

PALCO	DATA	HORA_INICIO	NOME_ARTISTICO	NOME_CIVIL
Palco Aquidauana	2026-09-18	20:00:00	Aurora do Pantanal	Aurora Ferreira Lima
Palco Aquidauana	2026-09-18	22:00:00	Dandara MC	Dandara Nogueira
Palco Aquidauana	2026-09-19	21:00:00	Guavira Beat	Gustavo Ovelar
Palco Cordilheira	2026-09-18	19:30:00	Guavira Beat	Gustavo Ovelar
Palco Cordilheira	2026-09-19	20:30:00	Aurora do Pantanal	Aurora Ferreira Lima
Palco Nascente	2026-09-19	18:00:00	Dandara MC	Dandara Nogueira

6 rows: confere com as 6 apresentações inseridas.

Equipe do show de abertura · idade dos artistas

S2: equipe técnica (N:M na prática)

```
-- S2) Equipe técnica do show de abertura (N:M na prática)
SELECT p.nome, t.funcao, e.turno
FROM escala_tecnica e
JOIN tecnico t ON t.cpf = e.cpf_tecnico
JOIN pessoa p ON p.cpf = t.cpf
WHERE e.nome_palco = 'Palco Aquidauana'
      AND e.data = '2026-09-18' AND e.hora_inicio = '20:00:00'
ORDER BY e.turno, p.nome;
```

NOME	FUNCAO	TURNO
Estevão Rocha Martins	Iluminação	montagem
Flávia Terena	Palco/Rigging	montagem
Caio Souza Prado	Som (PA)	show

S3: idade (atributo derivado)

```
-- S3) Idade dos artistas: o atributo DERIVADO calculado, não
armazenado
SELECT ar.nome_artístico,
      TIMESTAMPDIFF(YEAR, p.data_nascimento, CURDATE()) AS idade
FROM artista ar
JOIN pessoa p ON p.cpf = ar.cpf
ORDER BY idade DESC;
```

NOME_ARTISTICO	IDADE
Aurora do Pantanal	32
Guavira Beat	30
Dandara MC	26

A coluna que **não criamos** quando pessoa virou tabela (idade, derivado) vira TIMESTAMPDIFF aqui: calculada a cada consulta, nunca fica armazenada nem desatualizada.

PARA PRATICAR DEPOIS DA AULA

Desafios: leve a modelagem além do Festival

①

ABRE_PARA: auto-relacionamento

Um artista de abertura pode preceder outro artista na mesma apresentação. Isso é um relacionamento **recursivo** em ARTISTA: como o diagrama de Chen representa uma entidade se relacionando consigo mesma?

②

INGRESSO → {PISTA, CAMAROTE}

INGRESSO se especializa em PISTA (sem assento) e CAMAROTE (nº de mesa). É total ou parcial? Disjunta? Qual das 3 estratégias do slide 22 você escolhe, e por quê?

③

Gêneros musicais multivalorado

Cada ARTISTA pode tocar mais de um gênero musical. Mesma regra de telefone_pessoa (R6): qual a PK da nova tabela? Precisa de id sintético?

Para os três: **desenhe em Chen, aplique a regra, escreva o DDL.**

ENCERRAMENTO

Do minimundo ao banco rodando

Minimundo → diagrama de Chen → 7 regras de mapeamento → DDL comentado → carga na ordem certa → consultas de sanidade → constraints que barram a linha errada quando a modelagem falha.

Minimundo → Chen → 7 regras → DDL → carga → S1-S3 → constraints

Cada etapa devolve ao início: se a consulta não confere ou a constraint não dispara, o problema nasceu lá atrás, no modelo.

LEITURA DE REFERÊNCIA

ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados*. Cap. 3–4 (modelo Entidade-Relacionamento e Estendido) e Cap. 9 (mapeamento do modelo ER/EER para o modelo relacional).

